Software Architecture and Design Specification

Project: Vehicle Parking System (VPS)

Team: Team 10

Authors: Swapna, Shreya R D, Himani Nune, Atharv Sameer Sawarkar

Version: 1.0

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	17-09-2026	Team 10	Initial draft of architecture and design specification

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Vehicle Parking System (VPS) version 1.0. It translates the requirements in the VPS SRS v1.0 into components, data models, interfaces and flows that developers can implement and testers can verify against the VPS Software Test Plan.

1.2 Scope

Covers user registration and login, parking slot viewing (floor-wise), slot booking with vehicle registration, check-in and check-out, booking history, and administrator functions (add/remove slots, view active bookings, force checkout). Payment processing, parking sensors, cameras and physical gate hardware are out of scope.

1.3 Audience

Developers, QA engineers, security reviewers, course evaluators and future maintainers.

1.4 Definitions

Term	Meaning
VPS	Vehicle Parking System
SRS / STP / RTM	Software Requirements Specification / Software Test Plan / Requirements Traceability Matrix
ADR	Architecture Decision Record

Term	Meaning
ORM	Object-Relational Mapping
MVC	Model-View-Controller
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
RBAC	Role-Based Access Control

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture (components, pattern, technology, risks, security). Section 4 describes the detailed design (data model, sequence diagrams, API, error handling, UX). Diagrams are given as PlantUML source so they can be rendered at plantuml.com or in the PlantUML plugin and pasted into the final report.

2.2 Related Documents

- Software Requirements Specification: Vehicle Parking System v1.0 (12-09-2026)
- Software Test Plan: Vehicle Parking System v1.0 (16-09-2026)
- Requirements Traceability Matrix (maintained with the SRS and STP)

3. Architecture

3.1 Goals and Constraints

Goals

- Correctness: a slot is never double-booked, and slot status always matches active bookings (VPS-NF-003).
- Security: hashed passwords, session-based access control, admin-only routes (VPS-SR-001 to 005).
- Performance: dashboard loads within 2 seconds for up to 200 slots (VPS-NF-001).
- Maintainability: models, routes and templates are separated (VPS-NF-005).
- Usability: core actions within 3 clicks (VPS-NF-002).

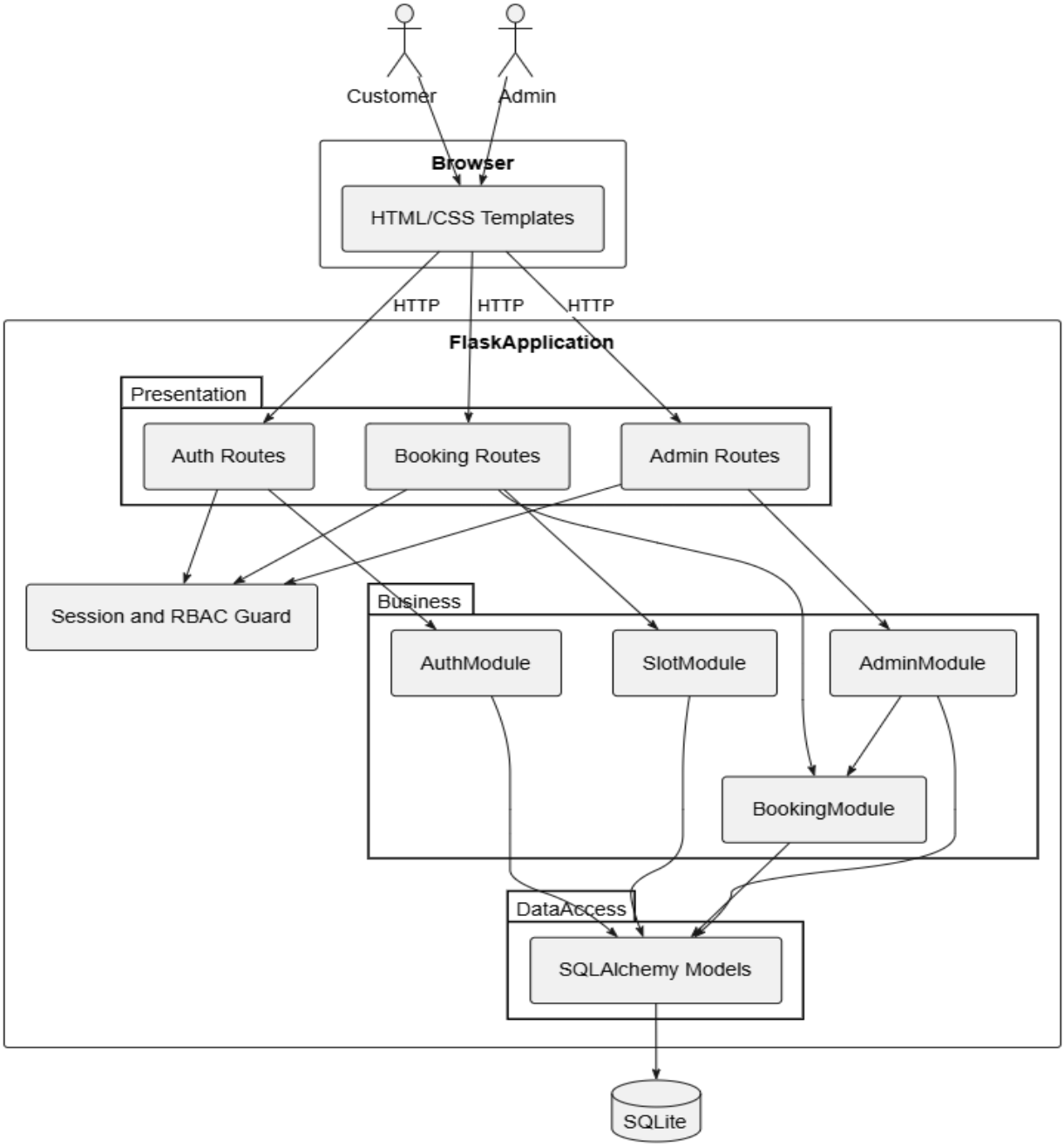
Constraints

- Python 3 with Flask, Flask-SQLAlchemy and SQLite.
- Session-based authentication only (no OAuth/SSO).
- Single facility per deployment; slots and floors configured manually by an admin.
- No payment gateway, sensors, cameras or gate hardware in this version.
- Course schedule: build and test between 15 and 23 September 2026.

3.2 Stakeholders and Concerns

Stakeholder	Concerns
Customer	Easy booking, accurate slot availability, private booking history
Admin / Parking staff	Slot inventory control, visibility of active bookings, ability to force checkout
Course Evaluator	Traceability to SRS, quality of design artefacts, security awareness
Developers	Modularity, simple setup, clear interfaces
QA Engineers	Testability, predictable behaviour, seedable test data

3.3 Component Diagram



3.4 Component Descriptions

Component	Responsibility

Component	Responsibility
Web UI (templates)	Server-rendered HTML/CSS pages: register, login, slot dashboard, booking form, history table, admin panel
Auth Routes / AuthModule	Registration, duplicate email check, password hashing (Werkzeug), login, logout, session creation and destruction
Session and RBAC Guard	Decorators such as login_required and admin_required that protect routes
SlotModule	Retrieves slots ordered by floor then slot number; checks availability
BookingModule	Creates bookings, registers or reuses vehicles, records check-in time, handles check-out and ownership checks
AdminModule	Add slot, remove available slot, list active bookings, force checkout
Models (ORM)	User, Vehicle, Slot, Booking entities mapped to SQLite tables
Database	Persistent storage of users, vehicles, slots and bookings

3.5 Chosen Architecture Pattern and Rationale

The VPS uses a **layered architecture** (Presentation, Business Logic, Data Access) implemented in Flask following an MVC-style split into routes, models and templates.

- **Why layered/MVC:** it gives clear separation of concerns, directly satisfies VPS-NF-005, keeps each layer independently testable (unit, integration and system levels in the STP) and is the natural structure for Flask.
- **Alternatives rejected:** microservices (too complex for a single-facility system with about 200 slots and a small team); a single-script monolith (poor maintainability and testability).

ADR-001: Layered Flask application

- Status: Accepted
- Decision: Use layered architecture with separate models, routes and templates.
- Consequences: Slight overhead of extra modules, but easier testing and future extension.

ADR-002: SQLite via Flask-SQLAlchemy

- Status: Accepted
- Decision: Use SQLite for the course deliverable, accessed only through the ORM.

- Consequences: Zero-configuration setup. Because all access goes through the ORM, the database can be swapped for PostgreSQL or MySQL in production without changing business logic. SQLite has limited write concurrency, acceptable for this scale.

ADR-003: Session-cookie authentication

- Status: Accepted
- Decision: Use server-side Flask sessions with a signed cookie; no OAuth/SSO (per SRS constraint).
- Consequences: Simple logout and invalidation; HTTPS recommended in production.

3.6 Technology Stack and Data Stores

Concern	Choice
Language / framework	Python 3, Flask
ORM	Flask-SQLAlchemy
Database	SQLite (single file)
Password hashing	Werkzeug generate_password_hash / check_password_hash
Frontend	Jinja2 templates, HTML, CSS
Testing	pytest / unittest, Flask test client
Transport	HTTP in development, HTTPS in production
Tools	PlantUML, draw.io, Git

3.7 Risks and Mitigations

Risk	Mitigation
Double booking due to concurrent requests	Check slot status and create booking in a single database transaction; commit atomically
Slot status and booking state become inconsistent	Update slot and booking in the same transaction; run consistency check (TC-Rel-01)
Unstable build or database errors	Smoke tests first; keep backup test database and seed script
Dashboard slows with many slots	Single ordered query, indexed floor and slot number; test with 200 slots (TC-Perf-01)
Limited time	Prioritise high-priority requirements; implement modules in order Auth, Slots, Booking, Admin

Risk	Mitigation
SQLite concurrency limits	Acceptable for one facility; ORM allows migration to PostgreSQL/MySQL

3.8 Traceability to Requirements

Requirement	Description	Module	Test Case
VPS-F-001	User registration	AuthModule	TC-Reg-01
VPS-F-002	Duplicate email validation	AuthModule	TC-Reg-02
VPS-F-003	Password hashing	AuthModule	TC-Auth-01
VPS-F-004	User authentication	AuthModule	TC-Auth-02
VPS-F-005	Display parking slots	SlotModule	TC-Slot-01
VPS-F-006	Floor-wise display	SlotModule	TC-Slot-02
VPS-F-007	Prevent booking occupied slot	SlotModule, BookingModule	TC-Slot-03
VPS-F-008	Book available slot	BookingModule	TC-Book-01
VPS-F-009	Register new vehicle	BookingModule	TC-Book-02
VPS-F-010	Reuse existing vehicle	BookingModule	TC-Book-03
VPS-F-011	Record check-in time	BookingModule	TC-Book-04
VPS-F-012	Customer check-out	BookingModule	TC-Checkout-01
VPS-F-013	Check-out authorisation	BookingModule, Guard	TC-Checkout-02
VPS-F-014	View booking history	BookingModule	TC-Hist-01
VPS-F-015	Admin add slot	AdminModule	TC-Admin-01
VPS-F-016	Admin remove available slot	AdminModule	TC-Admin-02
VPS-F-017	Admin view / force checkout	AdminModule	TC-Admin-03
VPS-NF-001	Dashboard under 2 s	SlotModule, Web UI	TC-Perf-01
VPS-NF-002	3-click navigation	Web UI	TC-UX-01

Requirement	Description	Module	Test Case
VPS-NF-003	Slot/booking consistency	BookingModule, Models	TC-Rel-01
VPS-NF-004	Session expiry on logout	AuthModule	TC-Sec-01
VPS-NF-005	Separation of models, routes, templates	Whole codebase	CR-01
VPS-SR-001	Secure password hashing	AuthModule	TC-Sec-02
VPS-SR-002	Admin route authorisation	Guard, AdminModule	TC-Sec-03
VPS-SR-003	Session invalidation	AuthModule	TC-Sec-01
VPS-SR-004	Input validation and sanitisation	All routes	TC-Sec-04
VPS-SR-005	Checkout by owner/admin only	BookingModule, Guard	TC-Checkout-02

3.9 Security Architecture

Threat modelling (STRIDE)

Threat	Example in VPS	Mitigation
Spoofing	Attacker logs in as another user	Password hashing, generic login error message, signed session cookie
Tampering	Modified form fields or cookie; SQL injection	Server-side validation, ORM parameterised queries, signed cookies, CSRF tokens
Repudiation	User denies a booking or admin denies forced checkout	Bookings store user, check-in and check-out timestamps; admin actions logged
Information disclosure	Viewing another user's history; plaintext passwords	Query bookings by session user id; hashed passwords; no sensitive data in logs; HTTPS in production
Denial of service	Flooding login or booking endpoints	Basic rate limiting (future: Flask-Limiter); small, indexed queries

Threat	Example in VPS	Mitigation
Elevation of privilege	Customer calls admin routes or checks out another user's booking	admin_required decorator (RBAC); ownership check on checkout

Security controls summary: hashed passwords (VPS-SR-001), RBAC on admin routes (VPS-SR-002), session cleared on logout (VPS-SR-003), input validation and ORM parameterisation (VPS-SR-004), owner/admin-only checkout (VPS-SR-005).

4. Design

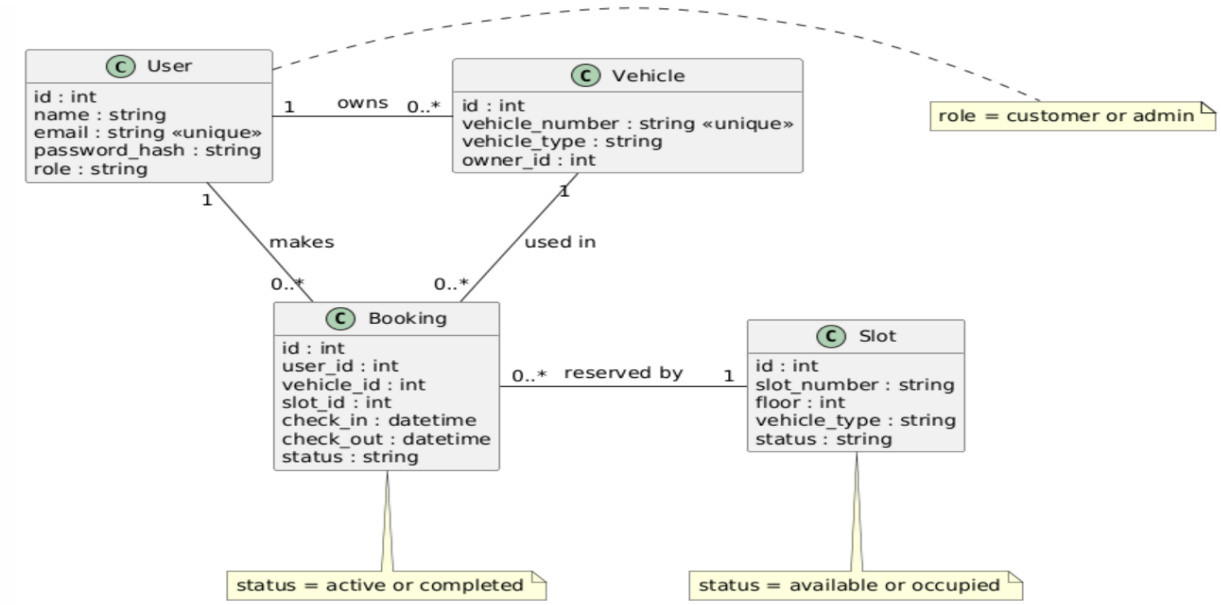
4.1 Design Overview

The VPS is a layered Flask application. Routes (controllers) receive HTTP requests, apply the session and role guards, and delegate to business modules. Business modules use SQLAlchemy models to read and write the database and return data to Jinja2 templates for rendering.

Suggested project structure

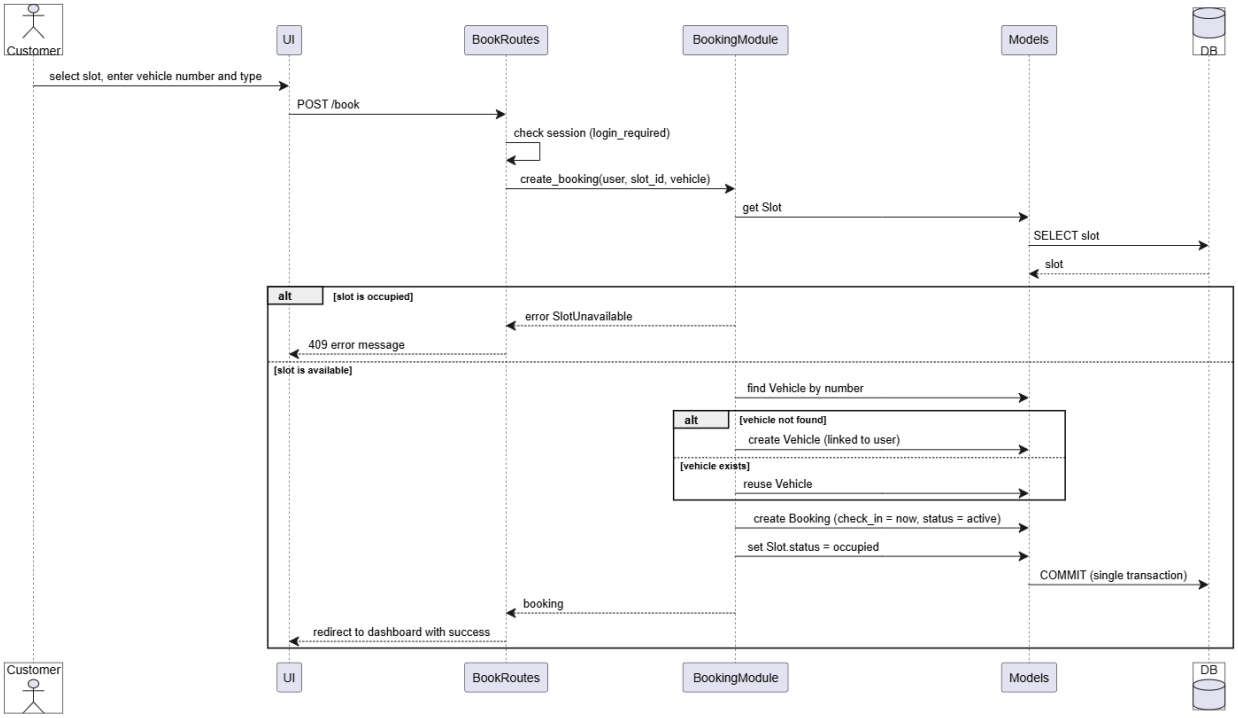
```
vps/
  app.py              # app factory, config
  models.py           # User, Vehicle, Slot, Booking
  routes/
    auth.py  booking.py  admin.py
  services/
    auth_service.py  slot_service.py  booking_service.py  admin_service.py
  templates/          # base, login, register, dashboard, book, history, admin
  static/             # css
  tests/
```

Data model

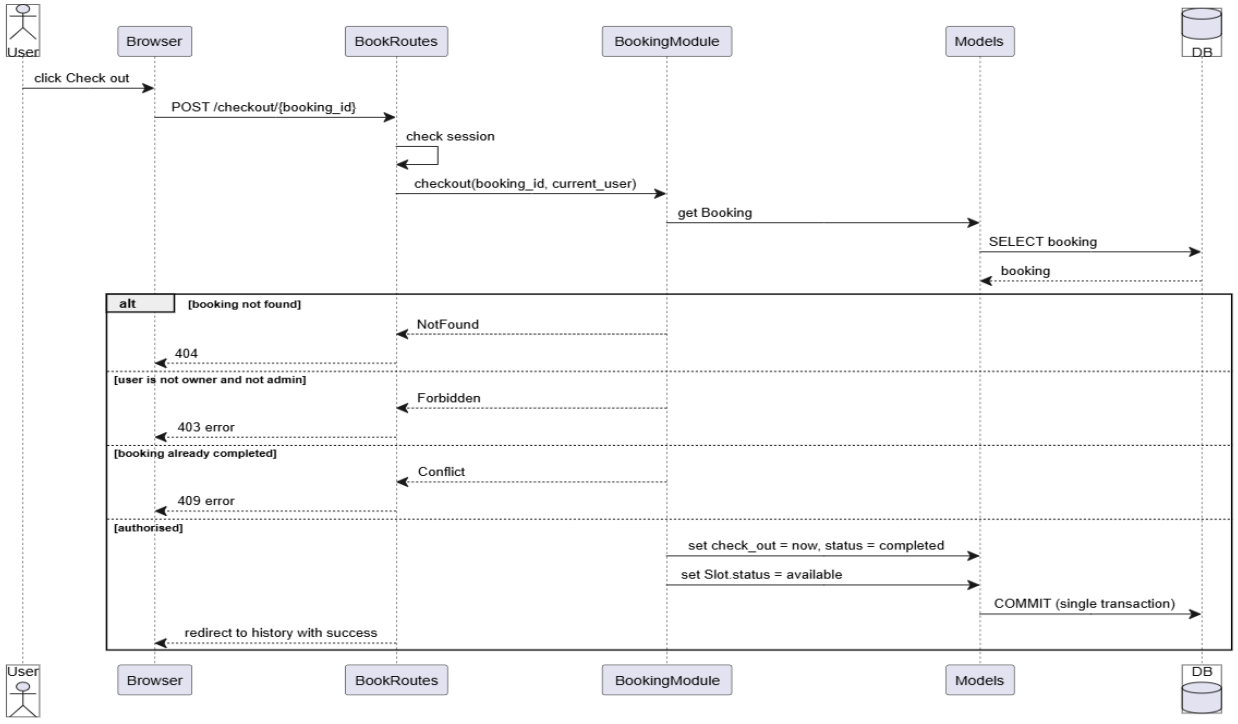


4.2 UML Sequence Diagrams

Flow 1: Book a parking slot (VPS-F-007 to VPS-F-011)



Flow 2: Check out a vehicle (VPS-F-012, VPS-F-013, VPS-F-017)



4.3 API Design

The VPS is server-rendered, so these are HTTP route interfaces that return HTML pages (or redirects). Requests are form-encoded. Interfaces are defined for the Auth, Booking and Admin components.

Auth component

Endpoint	Method	Access	Request	Success	Errors
/register	POST	Public	name, email, password	302 redirect to /login	400 invalid input, 409 duplicate email
/login	POST	Public	email, password	302 redirect to /dashboard, session started	401 invalid credentials
/logout	GET/POST	Logged in	none	302 to /login, session cleared	none

Booking component

Endpoint	Method	Access	Request	Success	Errors
/dashboard	GET	Logged in	none	200 slot list grouped by floor with status	302 to /login if not authenticated
/book	POST	Customer	slot_id, vehicle_number, vehicle_type	302 to /dashboard, booking created, slot occupied	400 invalid input, 404 slot not found, 409 slot occupied
/checkout/{booking_id}	POST	Owner or Admin	none	302 to /history, slot released	403 not owner, 404 not found, 409 already completed
/history	GET	Customer	none	200 own bookings with check-in/out and status	302 to /login

Admin component

Endpoint	Method	Access	Request	Success	Errors
/admin/slots	POST	Admin	slot_number, floor, vehicle_type	302 to /admin, new available slot	400 invalid input, 409 duplicate slot, 403 non-admin
/admin/slots/{id}/delete	POST	Admin	none	302 to /admin	409 slot occupied, 404 not found, 403 non-admin
/admin/bookings	GET	Admin	none	200 list of active bookings	403 non-admin
/admin/checkout/{booking_id}	POST	Admin	none	302 to /admin/bookings, slot released	404 not found, 409 already completed, 403 non-admin

Example: POST /book

- Request: slot_id=12, vehicle_number=KA01AB1234, vehicle_type=Car
- Success: booking record {booking_id, slot_id, check_in, status: active}
- Error 409: Slot is already occupied.

4.4 Error Handling, Logging and Monitoring

Error handling

- All user input is validated server-side (required fields, email format, vehicle number pattern, slot number uniqueness) before reaching the database (VPS-SR-004).
- Standard error pages and flash messages for 400, 401, 403, 404, 409 and 500; messages never reveal stack traces or whether an email exists on login failure.
- Database operations use transactions with rollback on failure so slot and booking state stay consistent (VPS-NF-003).

Logging

- Log events: login success/failure (email only), booking created, check-out, forced checkout (with admin id), slot added/removed, errors.
- Never log passwords, password hashes or session cookies.
- Each log line has timestamp, level, user id and action.

Monitoring (basic)

- Track booking and checkout failure counts from the log, dashboard response time, and results of the periodic consistency check (no occupied slot without an active booking).

4.5 UX Design

- Clean slot dashboard grouped by floor, using colour plus text labels (Available / Occupied) so status is not colour-only.
- Navigation bar with Dashboard, Book, History and Logout (Admin Panel for admins), so core actions are within 3 clicks (VPS-NF-002).
- Responsive layout for desktop and mobile browsers.
- Clear form labels, inline validation messages and success/error banners.
- Confirmation prompt before check-out and before slot removal.
- Readable font sizes and sufficient contrast.

4.6 Open Issues and Next Steps

- Payment and billing based on parking duration (excluded in v1.0).
- Filter slots by floor or vehicle type (listed as future enhancement in SRS).
- Automatic vehicle detection using sensors/cameras and gate control.
- Email/SMS booking confirmation and advance reservation.
- Migration to PostgreSQL/MySQL and HTTPS deployment for production.
- Rate limiting and account lockout after repeated failed logins.
- Decide handling of bookings left active for very long durations.

5. Appendices

5.1 Glossary

VPS, SRS, STP, RTM, ADR, ORM, MVC, RBAC, STRIDE, HTTPS, CSRF (see Section 1.4).

5.2 References

- IEEE 42010 (Architecture description)
- IEEE 830 / ISO 29148 (Requirements)
- OWASP Top 10 and OWASP ASVS
- NIST SP 800-160
- Flask and Flask-SQLAlchemy documentation
- VPS SRS v1.0 and VPS STP v1.0

5.3 Tools

PlantUML, draw.io, Flask, SQLite, Git, pytest.